workflow and core architecture

Building a custom LLM inference engine from scratch bridges low-level hardware memory management with cutting-edge ML serving architectures. The system drastically reduces the latency and memory footprint of large language models during text generation.

Core Architecture & Technical Stack

Tech Stack

- **Core Engine:** C++ (for memory management and execution loops), CUDA (optional/lightweight kernels).
- **ML Integration:** Python, PyTorch, Hugging Face `transformers` (for loading model weights).
- **Bindings & API:** `pybind11` (connecting C++ to Python), FastAPI.
- **Frontend (Optional):** React, WebSocket/Socket.io for real-time token streaming.

AI & Systems Concepts to Master

- **Autoregressive Generation:** Understanding how models calculate $P(x_n \vert{} x_1, ..., x_{n-1})$.
- **KV Caching & PagedAttention:** OS-level virtual memory paging applied to tensor allocations.
- **Continuous Batching:** Dynamic process scheduling algorithms for incoming prompt requests.
- **Speculative Decoding:** Tree-based heuristic search and rejection sampling.

Layman's Translation: The Three Big Problems

- **The KV Cache Problem:** To guess word #100, the AI has to re-read words 1-99. A "KV Cache" saves the mathematical summary of 1-99 so the AI only reads word #100.
- **The PagedAttention Problem:** Normally, the KV Cache demands massive, contiguous chunks of memory, leaving awkward unusable gaps (fragmentation). PagedAttention breaks the cache into tiny blocks and uses a map to track them, exactly like how an operating system manages computer RAM.
- **The Speculative Decoding Problem:** A massive, genius model (the target) takes 1 second to write 1 word. A tiny, dumb model (the draft) writes 5 words in 0.1 seconds. The tiny model guesses 5 words, and the genius model checks all 5 simultaneously. If the genius agrees, you just got 5 words for the price of 1.

Team Division (4-Person Architecture)

Role	Profile	Core Responsibilities
1. Systems Architect (You)	Advanced C++, OS/Networking Mechanics, Full-Stack	Write the C++ memory allocator mapping logical token sequences to physical blocks (PagedAttention). Implement the continuous batching scheduler to inject/evict API requests dynamically. Handle concurrency and row-level-locking equivalents for memory blocks.
2. ML Algorithm Lead	ML Research Oriented	Handle PyTorch integrations. Write the statistical rejection sampling algorithm that mathematically decides if the Target model accepts or rejects the Draft model's guessed tokens.
3. API & Middleware Dev	Beginner 1	Build the FastAPI layer. Use pybind11 to route HTTP requests from users into the C++ engine. Manage the queue of incoming text prompts.
4. Telemetry & UI Dev	Beginner 2	Build a React frontend visualizing real-time metrics: Tokens/Second, GPU Memory Usage, and a live highlighted text view showing which words were guessed by the draft model vs the target model.

Step-by-Step Implementation Flow

Phase 1: The Naive Baseline (Weeks 1-2)

- Load a small model (e.g., Llama-3-8B) in raw PyTorch.
-
- Implement a standard generation loop without any optimizations.
-
- Beginner 2 sets up the benchmarking script to record the baseline tokens-per-second and memory usage.
-

Phase 2: Continuous Batching & The Engine Shell (Weeks 3-4)

- Beginner 1 builds a FastAPI server that accepts multiple requests simultaneously.
-
- You build the C++ scheduler. Instead of waiting for Request A to finish completely before starting Request B, the engine evaluates generation step-by-step, inserting Request B into the compute pool the moment a slot opens.
-

Phase 3: Paged KV-Cache (Weeks 5-7)

- You implement the Block Table in C++. Divide the GPU memory into fixed-size blocks (e.g., 16 tokens per block).
-
- When a request comes in, the engine dynamically allocates physical blocks and updates the mapping table, ensuring zero memory fragmentation.
-
- The ML Lead maps the PyTorch attention mechanism to read from these fragmented blocks using your table.
-

Phase 4: Speculative Verification (Weeks 8-9)

- The ML Lead integrates a secondary, smaller draft model (e.g., Llama-3-1B).
-
- Create the sync loop: the Draft model generates a sequence tree of 4 tokens. The Target model runs a single forward pass over those 4 tokens.
-
- Implement the statistical verification rule to accept/reject the draft tokens.
-
- Beginners 1 & 2 wire the final output to the React dashboard, showcasing the real-time speedup.
-

What specific C++ or Python environment setup will you use to bridge the low-level memory management with the PyTorch ecosystem?

system architecture

Building a production-grade inference engine by November 1st requires a strict nine-week pipeline divided across C++ memory management, Python machine learning algorithms, and API middleware.

System Architecture: The C++ / Python Bridge

- **The Brain (Python/PyTorch):** Handles the actual neural network math and loads the model weights (e.g., Llama-3-8B) using Hugging Face.
-
- **The Engine (C++):** Manages the RAM/VRAM. Instead of letting Python handle memory (which is slow and fragmented), the C++ engine allocates physical blocks and tells Python exactly where to read and write.
-
- **The Bridge (`pybind11`):** A library that allows Python code to call your custom C++ functions as if they were native Python modules.
-

Layman's Translation: The Language Barrier
Python is slow but great at AI math. C++ is lightning fast at memory management but terrible for writing neural networks. Using `pybind11` creates a direct pipeline. When a user sends a prompt, Python catches it, immediately hands the scheduling and memory mapping over to C++, and C++ barks orders back at Python on how to execute the generation step efficiently.

9-Week Development Sprint (Sept 1 – Nov 1)

Week	Phase	Systems Architect (You)	ML Lead	Middleware & UI (Beginners 1 & 2)
1-2	**Baselines & Scaffolding**	Setup C++ project layout, configure CMake, and write basic memory structs.	Run Llama-3-8B locally in PyTorch; document baseline latency/VRAM.	Setup FastAPI backend and React boilerplate; build a basic chat UI.

3-4	**The Bridge & Scheduler**	Expose C++ structs to Python via pybind11. Write the Continuous Batching queue logic.	Route PyTorch inputs through the C++ batching scheduler instead of Python lists.	Connect FastAPI endpoints to the Python/C++ pipeline.
5-6	**Paged KV Cache**	Build the BlockAllocator class. Handle pointer arithmetic to map logical tokens to physical VRAM blocks.	Modify the PyTorch attention mechanism to read from the C++ generated block table.	Build WebSocket connections for real-time text streaming.
7-8	**Speculative Decoding**	Optimize C++ loops to handle multi-token verification arrays simultaneously.	Implement the draft model (1B) and write the statistical rejection sampling logic.	Add UI telemetry: graph tokens/sec and highlight drafted vs. verified words.
9	**Profiling & Launch**	Profile memory leaks; optimize C++ pointer references; finalize documentation.	Tune temperature/top-p settings for the draft model to maximize acceptance rate.	Stress-test the API with JMeter; deploy frontend.

This timeline ensures the beginners can work independently on web protocols while the complex algorithmic logic is isolated in the C++ and PyTorch environments.

Do you want to use standard CPU RAM for the C++ memory allocator to keep the initial build simple, or do you plan to write custom CUDA kernels to manage the GPU VRAM directly?